

Procesamiento y consulta de datos geoespaciales

Informe técnico del sistema distribuido y su despliegue continuo

Juan Guillermo Echeverri

Sebastián Botero Velásquez

Santiago Villamizar Mejía

Octubre de 2026

Repositorio: <https://github.com/sBoterino/big-data-geoespacial>

Resumen ejecutivo

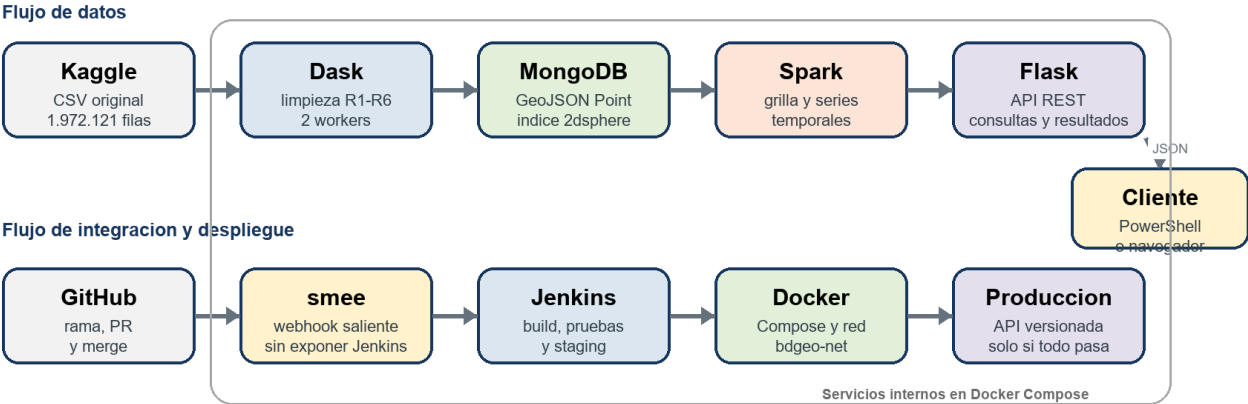
El sistema procesa el dataset NYC Motor Vehicle Collisions - Crashes con 1.972.121 registros. Dask ejecuta la limpieza distribuida, MongoDB almacena 1.741.828 documentos geoespaciales, Spark calcula agregaciones espaciales y temporales, Flask publica las consultas y Jenkins automatiza pruebas, staging y despliegue. La carga completa se reprodujo en un segundo computador y el pipeline impide desplegar cuando una prueba falla.

Resultado principal. La arquitectura satisface el procesamiento de más de un millón de registros, conserva GeoJSON válido y resultados verificables, y puede levantarse desde GitHub con Docker Compose y secretos locales.

1 Arquitectura del sistema

La solución separa el flujo de datos del flujo de entrega. Los servicios se comunican por la red interna bdgeo-net mediante nombres de servicio; los puertos publicados se usan solo desde el computador anfitrión. MongoDB, los datos descargados y Jenkins persisten en volúmenes Docker.

Arquitectura y flujos del sistema



Persistencia local: volumen MongoDB, volumen de datos Kaggle y volumen Jenkins.

Figura 1. Arquitectura, flujo de datos y flujo CI/CD.

Componente	Responsabilidad	Despliegue
Dask	Descarga, limpieza R1-R6 e ingesta por particiones	scheduler y 2 workers
MongoDB	Documentos, validación GeoJSON e índices	servicio con volumen
Spark	Agregaciones por grilla y tiempo	master y worker
Flask	API REST, validación y tiempos	Gunicorn versionado
Jenkins	Build, pruebas, staging y despliegue	perfil ci

2 Datos limpieza e ingesta

Se seleccionó NYC Motor Vehicle Collisions - Crashes porque aporta latitud, longitud, fecha, hora y defectos reales de calidad. La copia verificada el 1 de octubre de 2026 contiene 1.972.121 filas, 29 columnas y 420.704.526 bytes. Dask procesó seis particiones con dos workers y registró cada descarte en la primera regla incumplida.

Regla	Criterio	Descartados
R1	Coordenadas nulas o no numéricas	226.028
R2	Punto centinela (0,0)	4.115
R3	Fuera de rangos WGS84	106
R4	Válida globalmente pero fuera de NYC	44
R5	Fecha u hora inválida	0
R6	COLLISION_ID duplicado	0
Final	Documentos válidos insertados	1.741.828

Control de integridad. Los 230.293 descartes y los 1.741.828 documentos finales suman exactamente las 1.972.121 filas iniciales. La carga completa tardó 57,435 s en el equipo de desarrollo.

Idempotencia y persistencia

La colección `ingesta_meta` registra dataset, límite de filas, estado y total final. Una segunda ejecución compara esos metadatos con el conteo real y muestra Ingesta omitida si coinciden. `FORCE_RELOAD` permite una recarga explícita. El CSV y el Parquet permanecen en un volumen compartido, por lo que los builds no descargan nuevamente el archivo.

Modelo de documento

`COLLISION_ID` se usa como `_id`; fecha se guarda como BSON Date; location es un GeoJSON Point en orden [longitud, latitud]. MongoDB aplica un esquema de validación y mantiene los índices `_id`, `fecha_1` y `location_2dsphere`.

3 Consultas geoespaciales y API

La API Flask valida tipos, rangos y geometría antes de consultar MongoDB. Todas las operaciones aceptan límites, responden JSON, devuelven tiempo_ms y rechazan entradas inválidas con HTTP 400. El índice 2dsphere habilita las búsquedas sobre la superficie terrestre.

Método y ruta	Operación	Uso
GET /health	Ping y versión	Estado y evidencia del despliegue
GET /near	\$near	Vecinos ordenados por distancia
POST /within	\$geoWithin	Eventos dentro de un Polygon GeoJSON
GET /geonear	\$geoNear	Distancia y agrupación opcional
GET /spark-results	Lectura controlada	Catálogo de resultados Spark
GET /spark-results/<nombre>	Consulta de colección	Hotspots y series temporales

Pruebas sobre 1.741.828 documentos

Consulta	Resultado	Promedio caliente
\$near	100 documentos	13,493 ms
\$geoWithin	100 documentos	99,531 ms
\$geoNear simple	100 documentos	182,280 ms
\$geoNear agrupado	5 grupos	1.786,176 ms

Las mediciones calientes excluyen la primera ejecución. \$geoNear agrupado es más costoso porque calcula distancias para todos los eventos del radio antes de agrupar; la variante simple puede detenerse al completar el límite. /within recibe cualquier polígono válido, por lo que cambiar la zona no exige desplegar código.

4 Procesamiento y verificación con Spark

Spark lee desde MongoDB mediante el MongoDB Spark Connector. Un pipeline de proyección elimina campos innecesarios antes de transferir datos. La grilla regular usa celdas de 0,005 grados, aproximadamente 555 por 420 m en Nueva York, y puede modificarse con el parámetro --celda.

Salida	Contenido	Control
spark_grilla	n, heridos, muertos y polígono por celda	suma n = total
spark_hotspots	20 celdas de mayor concentración	ranking reproducible
spark_por_hora	conteo por hora	suma n = total
spark_por_dia_semana	conteo lunes a domingo	suma n = total
spark_por_mes	conteo mensual	suma n = total
spark_hora_borough	hora por borough	consistencia interna

Verificación independiente

verificar_resultados.py exige que grilla, hora, día y mes sumen el total de documentos y vuelve a contar el hotspot principal con \$geoWithin en MongoDB, con tolerancia de 0,5 %. En la semilla determinista, Spark obtuvo 57 y MongoDB 57. Sobre datos reales, Spark procesó 1.741.828 registros en 3.246 celdas; el hotspot principal tuvo 5.336 en Spark y 5.337 en MongoDB, dentro del margen permitido.

Semilla determinista

La colección eventos_semilla contiene 300 puntos de tres zonas de Nueva York. Los controles 120, 80 y 100 verifican conectividad, orden [longitud, latitud] y consultas espaciales sin depender de la carga completa. Cada build escribe resultados spark_semilla_* para no alterar las colecciones reales que consume la API.

5 Integración pruebas y despliegue continuo

El trabajo se integra mediante ramas y pull requests. Un merge a main genera un push; GitHub lo envía al canal smee y el cliente local lo reenvía a Jenkins por la red interna. Jenkins usa el Docker del host, pero no se publica en Internet.

Etapa	Comprobación principal
Checkout	Commit, autor y Jenkinsfile versionado
Construir imágenes	API, ingesta, Spark y servicios Compose
Pruebas unitarias	30 API, 5 ingesta y 4 Spark
Kaggle	Credencial y disponibilidad del dataset
Servicios e ingesta	Healthchecks e idempotencia en main
Integración	2 workers Dask y Spark-MongoDB
Procesamiento Spark	Semilla siempre; datos reales bajo condición
Staging	Imagen candidata y smoke tests 120/120/120
Despliegue	Solo main y después de aprobar todo

Bloqueo demostrado. Al ejecutar `FORZAR_FALLO=true`, pytest produce un fallo intencional, Jenkins omite las etapas posteriores y `/health` conserva la versión anterior. En los jobs de rama, staging usa eventos_semilla y el despliegue se omite, lo que evita modificar producción durante una validación.

Credenciales. MongoDB se inyecta desde mongo-root y Kaggle desde kaggle-api-token. En desarrollo se usa un .env ignorado por Git. Ningún token, contraseña, CSV o volumen se versiona.

6 Análisis comparativo Dask y Spark

Ambos motores ejecutaron la misma operación: leer el mismo Parquet, asignar cada punto a una celda de 0,005 grados, contar por celda y obtener el top 20. Se probaron uno y dos workers, con 1.741.828 filas y con una copia física de 17.418.280 filas. Cada combinación tuvo calentamiento y tres repeticiones válidas.

Tiempo de cálculo de la agregación por grilla



Figura 2. Tiempo de cálculo; media de tres repeticiones sin calentamiento.

Datos	Dask A / B	Spark A / B	Memoria observada Dask / Spark
x1	0,30 / 0,20 s	3,80 / 4,65 s	425-612 / 1.077-1.657 MiB
x10	2,36 / 1,39 s	4,51 / 5,12 s	491-698 / 1.121-1.816 MiB

Dask fue más rápido en las cuatro combinaciones. En la configuración A, la ventaja bajó de 12,7 veces en x1 a 1,9 veces en x10: el costo fijo de Spark pesa menos al crecer el volumen. Dask mejoró con dos workers; Spark empeoró porque ambos workers compartían el mismo PC y añadieron comunicación entre JVM. Los dos motores produjeron la misma celda top: n=5.336 en x1 y 53.360 en x10.

Limitaciones. La prueba se ejecutó en un solo computador, la memoria se muestreó cada aproximadamente 2 s y x10 repite filas. No se concluye que Dask sea universalmente superior ni que Spark vaya a superarlo; solo se reportan las condiciones medidas.

7 Decisiones y reproducibilidad

Decisión	Motivo y efecto
MongoDB local	Cumple Compose, evita el límite de Atlas y reduce dependencia de red
CI/CD desde el inicio	Reduce el riesgo de integrar Jenkins al final
Ingesta idempotente	Evita descargar y recargar casi dos millones de filas
Conector en imagen Spark	Versión fija y builds repetibles sin descarga por ejecución
Semilla de 300 puntos	Pruebas rápidas y resultados espaciales conocidos
Staging antes de promover	La misma imagen probada llega a producción
smee en vez de ngrok	Webhook saliente sin exponer Jenkins
Parquet físico x10	Comparación de volumen sin reutilización asimétrica de tareas

Reproducción en un segundo computador

Juan Guillermo clonó main en Windows con 16 GB de RAM, creó sus propios secretos y construyó los servicios sin copiar carpetas, imágenes, volúmenes ni datos. Verificó dos workers Dask, el conector Spark-MongoDB, la carga completa, la segunda ingesta omitida, RESULTADO: OK de Spark, /near y hotspots. La incidencia inicial fue Docker Desktop instalado pero con el motor apagado; abrirlo resolvió el problema.

Arranque desde cero

- Clonar el repositorio y copiar .env.example a .env.
- Definir contraseña MongoDB, token Kaggle y límites de memoria.
- Ejecutar docker compose up -d --build y comprobar /health.
- Configurar Jenkins con mongo-root, kaggle-api-token y el job bdgeo-main.
- Activar smee, hacer merge mediante PR y observar el pipeline.

8 Conclusiones

- El sistema procesó 1.972.121 filas, justificó 230.293 descartes y almacenó 1.741.828 documentos GeoJSON con índice 2dsphere.
- Dask resolvió la ingesta con dos workers e idempotencia; Spark generó agregaciones espaciales y temporales verificadas contra MongoDB.
- La API ofrece consultas parametrizadas, validación y tiempos sobre datos reales, además de resultados precalculados de Spark.
- El pipeline automatiza pruebas, integración, staging y despliegue; una prueba fallida conserva la versión anterior.
- En el hardware medido, Dask fue más rápido y usó menos memoria observada; la diferencia se redujo con diez veces más filas.
- La reproducción en otro computador confirmó que el README, Compose y los secretos locales bastan para levantar la solución.

Trabajo futuro

La evaluación puede ampliarse con datos geográficamente diversos, medición de memoria de mayor frecuencia, varias máquinas físicas, pruebas de carga concurrente sobre la API y una visualización Leaflet. Estas extensiones no son necesarias para validar los objetivos actuales.

Referencias y evidencias

1. Repositorio. <https://github.com/sBoterino/big-data-geoespacial>
2. Dataset. <https://www.kaggle.com/datasets/muzammilrizvi1/motor-vehicle-collisions-crashes>
3. Carga completa. docs/evidencias/gate4_carga_completa_2026-10-04.md
4. Consultas reales. docs/evidencias/fase5_datos_reales_2026-10-04.md
5. Validación Spark. docs/evidencias/fase6_logica_local_2026-10-04.md
6. Benchmark. docs/evidencias/fase8_benchmark_2026-10-05.md y benchmark/resultados.csv
7. Reproducibilidad. docs/evidencias/fase9_reproducibilidad_juan_2026-10-07.md
8. MongoDB Geospatial Queries. <https://www.mongodb.com/docs/manual/geospatial-queries/>
9. Docker Compose. <https://docs.docker.com/compose/>
10. Apache Spark. <https://spark.apache.org/docs/latest/>